Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

Proyecto 3 – Introducción a la Programación y Computación 2

202308221 – Jose Juan Laynez Zapeta

Resumen

El siguiente documento presenta la información técnica y arquitectónica del Proyecto 3 del curso de Introducción a la Programación y Computación 2.

Este proyecto consiste en el desarrollo de un sistema simulador de la Superintendencia de Administración Tributaria (SAT) para la autorización de Documentos Tributarios Electrónicos (DTE).

Con este proyecto, se pretende aplicar diversos temas de programación como la arquitectura de software en capas (API REST y cliente web), programación orientada a objetos, persistencia de datos mediante archivos JSON con control de concurrencia, validación de reglas de negocio (cálculo de IVA, validación de NIT) y generación de reportes estadísticos y gráficos.

Palabras clave

Programación Orientada a Objetos, API REST, DTE, SAT, .NET 8, Concurrencia, JSON

Abstract

The following document presents the technical and architectural information of Project 3 for the Introduction to Programming and Computing 2 course.

This project consists of developing a simulator system for the Tax Administration Superintendence (SAT) to authorize Electronic Tax Documents (DTE).

This project aims to apply various programming topics such as layered software architecture (REST API and web client), object-oriented programming, data persistence using JSON files with concurrency control, business rule validation (VAT calculation, NIT validation), and the generation of statistical and graphical reports.

Keywords

Object Oriented Programming,, API REST, DTE, SAT, .NET 8, Concurrency, JSON.

Introducción

A continuación, se presentará información detallada acerca del proyecto. Este consiste en un sistema diseñado para simular los procesos de la SAT, brindando una solución robusta para el registro de contribuyentes (RTU), la autorización de facturas electrónicas y la generación de reportes estadísticos y de cumplimiento.

Para lograr este objetivo, fue necesario trabajar con conceptos fundamentales de la ingeniería de software. Se implementó una arquitectura desacoplada utilizando .NET 8, donde el backend expone servicios mediante el protocolo HTTP y el frontend, desarrollado con Razor Pages, actúa como cliente consumidor. Un pilar fundamental fue el uso estricto de la Programación Orientada a Objetos (POO) para modelar entidades como Factura y Contribuyente, y la creación de Tipos de Datos Abstractos (TDA) apoyados en las colecciones nativas de C# para gestionar el estado de la aplicación de manera eficiente. Asimismo, se dio cumplimiento al requisito de utilizar expresiones regulares para la extracción precisa de fechas dentro de cadenas de texto no estructuradas, y se implementó un mecanismo de persistencia en archivos JSON protegido por bloqueos de concurrencia (lock) para garantizar la integridad de los datos ante peticiones simultáneas.

Desarrollo del tema

El proyecto fue elaborado con base en los siguientes aspectos fundamentales:

1. Arquitectura de Software y Programación Orientada a Objetos:

   El diseño del sistema se basó en el principio de Responsabilidad Única (SRP), separando claramente las preocupaciones del software en capas distintas:

   - Capa de Controladores (SatController): Actúa como el punto de entrada de las peticiones HTTP. Su única responsabilidad es validar la estructura básica de la solicitud, extraer los parámetros y delegar la lógica de negocio a los servicios correspondientes, devolviendo respuestas estandarizadas en JSON.
   - Capa de Servicios (AutorizacionService, ConsultaService, RtuService): Contienen el núcleo de la lógica de negocio. Por ejemplo, AutorizacionService es el encargado de evaluar cada regla fiscal de una factura.
   - Capa de Modelos (Factura, Contribuyente, CorrelativoDiario): Clases que encapsulan el estado y los datos, utilizando propiedades con get y set para garantizar el encapsulamiento.
   - Capa de Utilidades (NitValidator, ParserFecha, JsonLector): Clases estáticas que proveen funciones puras y reutilizables, como el cálculo del dígito verificador o el parseo de fechas, sin mantener estado interno.

2. Procesamiento de JSON y expresiones regulares:

   El sistema está diseñado para consumir mensajes JSON tal como se especifica en el enunciado. Para lograr una extracción robusta de los datos, se implementó la clase JsonLector, que navega el árbol de JsonElement de manera insensible a mayúsculas/minúsculas, permitiendo obtener valores numéricos o de texto de forma segura.

Un requisito específico del proyecto fue el uso de expresiones regulares. Dado que el campo TIEMPO en el JSON puede contener texto adicional (ej. "Guatemala, 15/01/2025 15:25 hrs."), se implementó una expresión regular en la clase ParserFecha para aislar únicamente la información útil:

```
(?<![0-9])([0-9]{1,2})/([0-9]{1,2})/([0-9]{4})\s+([0-9]{1,2}):([0-9]{2})
```

Esta expresión captura de forma segura el día, mes, año, hora y minuto, descartando cualquier texto circundante, y construye un objeto DateTime válido, manejando excepciones de formato (ArgumentOutOfRangeException) para marcar la factura con el error DatosInvalidos si la fecha es imposible (ej. 30 de febrero).

3. Algoritmo de validación de NIT:

   La validación del NIT es un componente crítico. Se implementó el algoritmo oficial de la SAT en la clase NitValidator. El proceso sigue estos pasos estrictos:

   Se toma el cuerpo del NIT (todos los dígitos excepto el último, que es el verificador).
   Se itera de derecha a izquierda, multiplicando cada dígito por un factor que inicia en 2 y se incrementa en 1 para cada posición.
   Se suman todos los resultados de las multiplicaciones.
   Se obtiene el residuo de esta suma dividida entre 11 (suma % 11).
   Se resta este residuo de 11.

   Finalmente, se calcula el módulo 11 de ese resultado. Si el valor final es 10, el dígito verificador debe ser la letra 'K'; de lo contrario, es el número resultante.
   Este algoritmo garantiza que solo se procesen NITs estructuralmente válidos, rechazando inmediatamente aquellos que no cumplan con la regla matemática.

4. Lógica de negocio y generación de correlativos:

   Al procesar una solicitud de autorización, el sistema evalúa múltiples reglas de negocio:

- Existencia en el RTU: Se verifica que tanto el NIT emisor como el receptor existan en el diccionario del RTU.
- Cálculo de Montos: Se valida que el iva sea exactamente el resultado de redondear(VALOR * 0.12, 2) y que el total sea la suma aritmética de valor + iva. Cualquier desviación, por mínima que sea, marca la factura con los errores IvaMalCalculado o TotalMalCalculado.
- Control de Duplicados: Se utiliza un HashSet<string> llamado ReferenciasAprobadas.

  Esta estructura de datos ofrece una búsqueda en tiempo constante O(1), lo que la hace altamente eficiente para verificar si una referencia (de máximo 40 caracteres) ya fue

autorizada previamente, evitando la duplicidad de facturas.

Si una factura supera todas las validaciones, se le asigna un Código de Aprobación. Este se genera concatenando la fecha actual en formato yyyyMMdd con un correlativo de 8 dígitos (ej. 00000001). El sistema mantiene un objeto CorrelativoDiario que resetea este contador a 1 cada vez que la fecha del sistema cambia, cumpliendo estrictamente con el requisito del enunciado.

5. Persistencia de datos y manejo de concurrencia:

A diferencia de los proyectos anteriores que usaban XML, este proyecto requiere persistencia en archivos JSON (rtu.json, facturas.json, correlativo.json). Dado que una API REST puede recibir múltiples peticiones simultáneas, existe el riesgo de condiciones de carrera (race conditions) al leer y escribir en estos archivos.

Para solucionar esto, se implementó un mecanismo de exclusión mutua en la clase AlmacenDatos. Se declara un objeto privado Lock y se utiliza la sentencia lock (_almacen.Lock) en todos los métodos que modifican o leen el estado compartido. Esto garantiza que solo un hilo de ejecución pueda acceder a la estructura de datos en un momento dado, previniendo la corrupción de archivos o la asignación de códigos de aprobación duplicados.

6. Generación de reportes y frontend:

El frontend, desarrollado con ASP.NET Core Razor Pages, consume la API para presentar la información. Se implementaron consultas complejas utilizando LINQ (Language Integrated Query) para agrupar y filtrar datos. Por ejemplo, para el "Resumen de IVA por fecha y NIT", el sistema filtra las facturas aprobadas de la fecha dada, agrupa los resultados por NIT y calcula las sumatorias de IVA emitido y recibido.

Adicionalmente, para cumplir con el requisito de generación de PDF, se integró la librería QuestPDF. Esta permite diseñar documentos de manera fluida y tipográfica, transformando los objetos de reporte (como EstadisticasResultado) en archivos PDF descargables con tablas formateadas y encabezados profesionales.

7. 4

Un aspecto fundamental en el desarrollo de esta solución fue la adecuada abstracción de datos y la implementación de mecanismos robustos para el manejo de errores.

Aunque el enunciado permite el uso de colecciones nativas de C#, el diseño se centró en encapsular estas estructuras dentro de Tipos de Datos Abstractos (TDA) de alto nivel, siendo el AlmacenDatos el principal, el cual actúa como una fachada unificada para toda la capa de persistencia. Internamente, este TDA orquesta un Dictionary para garantizar búsquedas de NIT en tiempo constante O(1), un List<Factura> para el almacenamiento secuencial de transacciones, y un HashSet<string> dedicado exclusivamente al control de unicidad de las referencias, optimizando así la validación de duplicados sin necesidad de recorrer toda la colección linealmente.

Paralelamente, la robustez del sistema se cimentó en un manejo exhaustivo de excepciones y validaciones defensivas. Dado que el API está expuesta a ser consumida por múltiples clientes, se implementaron filtros estrictos en cada endpoint.

Por ejemplo, el método de lectura de payloads captura explícitamente las JsonException para evitar que un cuerpo de solicitud malformado derribe el proceso de la aplicación, devolviendo en su lugar un código de estado HTTP 400 (Bad Request) con un mensaje descriptivo. Del mismo modo, el procesamiento de fechas mediante expresiones regulares no solo extrae el patrón dd/mm/yyyy hh24:mi, sino que envuelve la conversión a DateTime en un bloque de control para interceptar ArgumentOutOfRangeException (como un día 31 en un mes de 30 días), catalogando estos casos silenciosamente como DatosInvalidos en el registro de la factura.

Esta combinación de abstracción de datos eficiente y programación defensiva garantiza que el simulador de la SAT opere con alta disponibilidad y consistencia, incluso ante entradas erróneas o inesperadas.

Dificultades encontradas:

La mayores dificultades del proyecto fueron con los endpoints, ya que previo a esto no tenia mucha experiencia con estos, asi como obtener los resultados deseados con estos.

A la hora de ver el enunciado, no tenia una gran noción de donde comenzar, asi que investigando, note que lo mejor seria crear el backend con los endpoints, para luego solo obtener los datos desde el frontend.

Conclusiones

Después de completar este proyecto, puedo destacar varias conclusiones importantes:

Después de completar este proyecto, se pueden destacar varias conclusiones importantes:

1. **Dominio de la POO y Arquitectura en Capas:** La separación estricta entre controladores, servicios y modelos permitió un código limpio, escalable y fácil de depurar. Cada clase tiene una responsabilidad única, lo que facilita el mantenimiento y la prueba de componentes aislados.

2. **Importancia de la Concurrencia en Sistemas Web**: Implementar el mecanismo de lock en la capa de persistencia fue crucial. Se comprendió que, en un entorno HTTP sin estado, proteger los recursos compartidos (archivos JSON) es obligatorio para garantizar la integridad de los datos y la consistencia de los correlativos.

3. **Precisión en la Validación de Datos**: El desarrollo del algoritmo de validación del NIT y el uso de expresiones regulares para el parseo de fechas reforzó la importancia de validar exhaustivamente los datos de entrada antes de procesarlos.

4. **Eficiencia en Estructuras de Datos:** El uso de HashSet para el control de referencias duplicadas y Dictionary para el RTU demostró cómo la elección correcta de un Tipo de Dato Abstracto (TDA) impacta directamente en el rendimiento del sistema, ofreciendo búsquedas en tiempo constante.

5. **Integración de Herramientas de Reporte:** La integración de QuestPDF en el ecosistema .NET mostró la potencia de generar documentos profesionales directamente desde el código, sin depender de servicios externos, cumpliendo con los requisitos de presentación de informes de la SAT.

6. **Gestión de Versiones:** El cumplimiento de los 4 releases en GitHub obligó a mantener un historial de cambios ordenado, facilitando la identificación de errores y la estabilización del código hacia la versión final.

Referencias bibliográficas

Joyanes Aguilar, L. (2008). Fundamentos de Programación: Algoritmos, Estructura de Datos y Objetos. McGraw-Hill Education.

Microsoft. (2026). Documentación oficial de C# y .NET 8. Disponible en: https://docs.microsoft.com/es-es/dotnet/csharp/

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (REST). University of California, Irvine.

Superintendencia de Administración Tributaria (SAT). (2026). Especificaciones técnicas para la autorización de Documentos Tributarios Electrónicos (DTE). Guatemala.

Material de clase del curso Introducción a la Programación y Computación 2, USAC, 2do semestre 2026.

Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

Extensión:

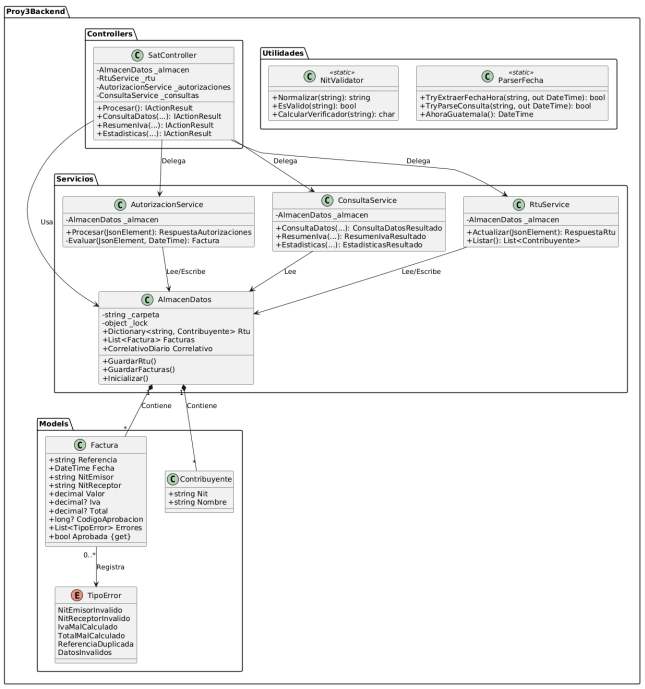


Diagrama de clases del Backend. *Fuente: Elaboración propia con plantUML.*

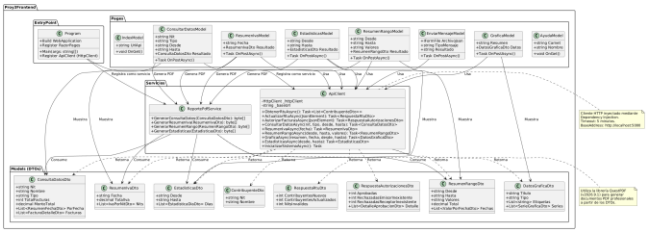


Diagrama de clases del frontend. *Fuente: Elaboración propia con PlantUML.*

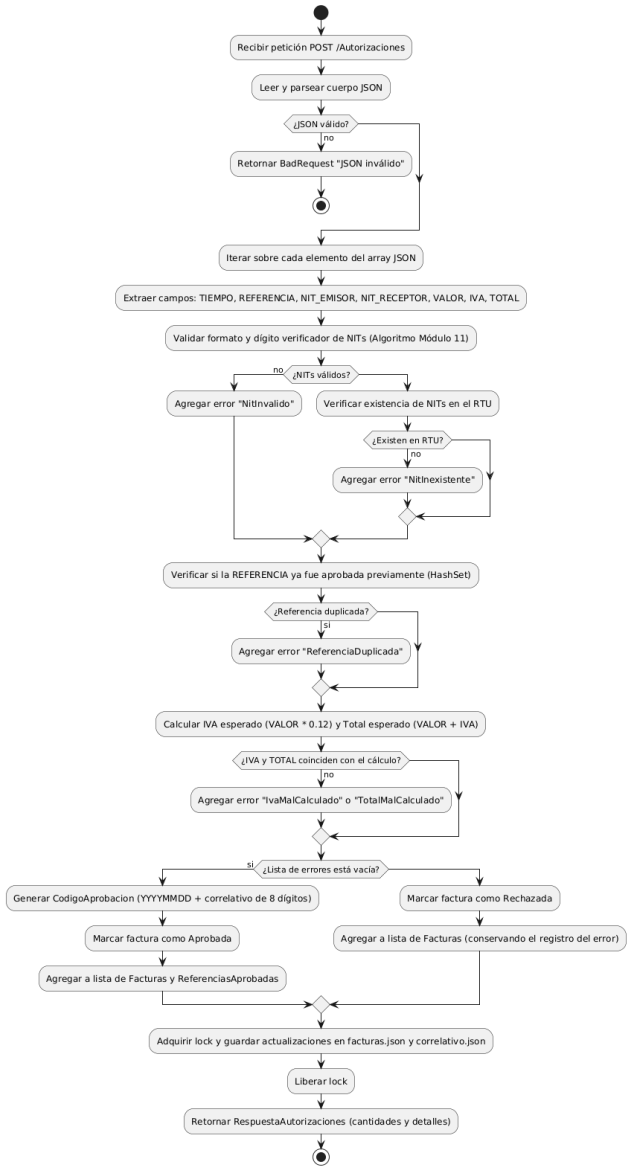


Diagrama de actividades.
Fuente: Elaboración propia con PlantUML.

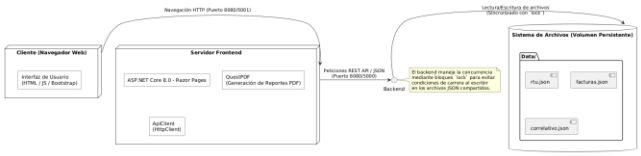
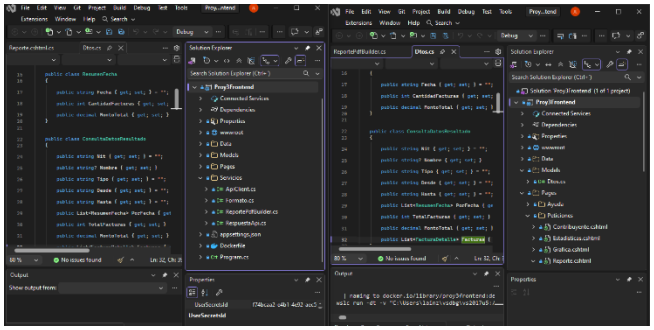


Diagrama de despliegue
Fuente: Elaboración propia con PlantUML.

Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026



Programa en funcionamiento
Fuente: Elaboración propia, a través de Microsoft Edge



Parte del código fuente, de ambos el backend y frontend
Fuente: Elaboración propia, a través de Visual Studio Community